OOP EXPERIMENT-5

NAME-ANMOL

SAP-590011794

BATCH-20

DATE-25 FEB 2026

SUBMITTED TO- PROF. Kalluri Shareef Babu

Theory

Method Overloading

Same method name is defined with different parameter lists (different number, type, or order of arguments).

Compile-time polymorphism is demonstrated through overloaded methods like:

calculate( ), area( ), volume( ), display( ).

Overload resolution is done by the compiler based on method signature.

Constructor Overloading

Multiple constructors in the same class initialize objects in different ways.

Default constructor initializes with default values.

Parameterized constructor initializes with user-provided values.

Demonstrated in classes like Employee andStudent style programs.

Constructor Chaining

this() is used inside one constructor to call another constructor of the same class.

Helps avoid duplicate initialization logic.

Must be the first statement in the constructor.

Default Constructor Concept

If no constructor is written, Java provides a default no-argument constructor.

Instance variables receive default values (e.g., int -> 0, reference -> null).

Once a custom constructor is defined, implicit default constructor is not provided.

Static Variables

static variables belong to the class, not individual objects.

Shared memory: all objects see the same static value.

Updates from one object are reflected for all objects.

Used for class-level counters like total object count.

Instance Variables

Instance variables belong to each object separately.

Each object keeps its own copy of instance state.

Changing one object's instance fields does not change another object's fields.

Static Methods and Access Rules

static methods are callable using class name.

A static method cannot directly access non-static (instance) members.

Reason: static context has no this reference to a specific object.

Blocks and Execution Order

static block executes once when class is loaded.

instance block executes each time an object is created, before constructor body.

constructor executes after instance block during object creation.

main() starts program execution.

Method Call Ambiguity in Overloading

Ambiguity occurs when compiler finds multiple overloaded methods equally suitable.

Example case: argument list that can match two methods via numeric conversion.

Resolved by explicit casting or using clearly typed arguments.

‘this’ Keyword

this.variable distinguishes instance field from parameter with same name.

this( ) invokes another constructor in same class.

Input Handling Used

Scanner (nextInt, nextDouble) for numeric keyboard input.

BufferedReader + InputStreamReader with readLine() for text input.

Integer.parseInt() converts string input to int.

Math Utility Functions

Math.sqrt() is used in geometric area computation.

Formula-based calculations are implemented through methods.

Output Functions

System.out.print() and System.out.println() are used for formatted console output.

Values are displayed from object methods and main() to show behavior.

Object Creation and Method Invocation

Objects are created using new.

Instance methods are called through object references.

Static members are accessed using class name or object reference

Exe5_1

```java
//Write a Java program that overloads a method calculate() to perform:
//- Sum of two integers
//- Sum of three integers
//- Sum of two double values
//- Concatenation of two strings
//Display the result of each method call from main().

class Overload
{
    void calculate(int a,int b)
    {
        System.out.println("Sum of two integers: "+(a+b));
    }
    void calculate(int a,int b,int c)
    {
        System.out.println("Sum of three integers: "+(a+b+c));
    }
    void calculate(double a, double b)
    {
        System.out.println("Sum of two double values: " +(a+b));
    }
    void calculate(String str1, String str2)
    {
        System.out.println("Concatenation of two strings: "+str1+str2);
    }
}

public class exe5_1
{
    public static void main(String args[])
    {
        Overload obj=new Overload();
        obj.calculate(17,19);
        obj.calculate(5,7,11);
        obj.calculate(2.96,5.67);
        obj.calculate("hell","o");
    }
}
```

```
OUTPUT    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe5_1.java
PS C:\Users\<Anmol> > java exe5_1
Sum of two integers: 36
Sum of three integers: 23
Sum of two double values: 8.629999999999999
Concatenation of two strings: hello
PS C:\Users\<Anmol> >
```

Observation: The program demonstrates the use of overloaded methods to perform various arithmetic operations and string concatenation on different data types.

Exe5_2

```java
// Create a class Employee with instance variables: name, id, salary.
// - Default constructor initializes with fixed values.
// - Parameterized constructor initializes with user-provided values.
// Create two objects and display employee details.
class employee
{
    String name;
    int id,salary;
    employee()
    {
        name="Rahul";
        id=590014725;
        salary=80000;
    }
    employee(String name,int id,int salary)
    {
        this.name=name;
        this.id=id;
        this.salary=salary;
    }
    void display()
    {
        System.out.println("Employee Name: "+name);
        System.out.println("Employee ID: "+id);
        System.out.println("Employee Salary: "+salary);
    }
```

```java
}
public class exe5_2
{
    public static void main(String args[])
    {
        employee e1=new employee();
        employee e2=new employee("Anmol Thapliyal",590011794,85000);
        e1.display();
        e2.display();
    }
}
```

```
OUTPUT 61    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe5_2.java
PS C:\Users\<Anmol> > java exe5_2
Employee Name: Rahul
Employee ID: 590014725
Employee Salary: 80000
Employee Name: Anmol Thapliyal
Employee ID: 590011794
Employee Salary: 85000
PS C:\Users\<Anmol> >
```

Observation: The program demonstrates the use of constructors to initialize object properties and display employee details using a method.

Exe5_3

```java
// Create a class Counter with:
// - static variable count
// - instance variable id
// Increment both variables inside constructor.
// Create 3 objects and display values of count and id for each object.
// Explain the output.
class Counter
{
    static int count;
    int id;
    Counter()
    {
        count++;
        id=count;
    }
```

```java
}
public class exe5_3
{
    public static void main(String args[])
    {
        Counter c1=new Counter();
        Counter c2=new Counter();
        Counter c3=new Counter();
        System.out.println("Counter 1: count="+Counter.count+", id="+c1.id);
        System.out.println("Counter 2: count="+Counter.count+", id="+c2.id);
        System.out.println("Counter 3: count="+Counter.count+", id="+c3.id);
    }
}
```

```
PS C:\Users\<Anmol> > javac exe5_3.java
PS C:\Users\<Anmol> > java exe5_3
Counter 1: count=3, id=1
Counter 2: count=3, id=2
Counter 3: count=3, id=3
PS C:\Users\<Anmol> >
```

Observation: The static variable 'count' is shared among all instances of the Counter class, so it increments with each new object created. The instance variable 'id' is unique to each object and is assigned the current value of 'count' at the time of object creation. Therefore, all objects will have the same 'count' value, but different 'id' values corresponding to their creation order.

Exe5_4

```java
// Create a class that has one instance variable and one static variable.
// Create a static method trying to access both variables.
// Observe and fix the compilation error.
// Explain why static methods cannot access instance variables.
class issta
{
    int a;
    static int b;
    static void display()
    {
        //System.out.println("Instance variable a = "+a); // This line would
cause a compilation error
```

```java
        System.out.println("Static variable b="+b);

    }
    void view()
    {
        System.out.println("Instance variable a="+a);
        System.out.println("Static variable b="+b);
    }
}
public class exe5_4
{
    public static void main(String args[])
    {
        issta.display();
        issta obj=new issta();
        obj.view();
    }
}
```

```
PS C:\Users\<Anmol> > javac exe5_4.java
PS C:\Users\<Anmol> > java exe5_4
Static variable b=0
Instance variable a=0
Static variable b=0
PS C:\Users\<Anmol> >
```

Observation: Static methods cannot access instance variables because they belong to the class rather than any specific instance. Instance variables require an object of the class to be accessed, while static methods can be called without creating an object. Therefore, static methods can only directly access static variables and cannot reference instance variables without an object.

Exe5_5

```java
// Write a program containing:
// - Static block
// - Instance block
// - Constructor
// - main() method
// Print statements in each block and observe execution order.
class exe5_5
{
    static
    {
        System.out.println("This is a static block.");
```

```
    }
    {
        System.out.println("This is an instance block.");
    }
    exe5_5()
    {
        System.out.println("This is a constructor.");
    }
    public static void main(String args[])
    {
        exe5_5 obj=new exe5_5();
        System.out.println("This is the main method.");
    }
}
```

```
PS C:\Users\<Anmol> > javac exe5_5.java
PS C:\Users\<Anmol> > java exe5_5
This is a static block.
This is an instance block.
This is a constructor.
This is the main method.
PS C:\Users\<Anmol> >
```

Observation: The execution order is as follows: First, the static block is executed when the class is loaded. Then, when an object of the class is created, the instance block is executed followed by the constructor. Finally, the main method is executed. Therefore, the output will be:

Exe5_6

```
// Write a program with:
// void display(int a, float b)
// void display(float a, int b)
// Call display(10, 10);
// Does it compile? Explain why ambiguity occurs.
class Ambiguity
{
    void display(int a,float b)
    {
        System.out.println("Method with int and float parameters called:
"+a+", "+b);
    }
    void display(float a,int b)
```

```
    {
        System.out.println("Method with float and int parameters called:
"+a+", "+b);
    }
}
public class exe5_6
{
    public static void main(String args[])
    {        Ambiguity obj=new Ambiguity();
        //obj.display(10, 10); // This line will cause a compilation error due
to ambiguity
        obj.display(10.6f,10);
        obj.display(10,10.8f);
        System.out.println("The call obj.display(10, 10) is ambiguous because
both methods can be called with the given arguments. The compiler cannot
determine which method to call since both methods can accept the arguments (10
can be treated as an int or a float). To resolve this ambiguity, you can
either cast the arguments to the desired types or provide different argument
values that clearly match one of the method signatures.");
    }
}
```

```
OUTPUT 66   TERMINAL   PORTS                                   pwsh - exe5
PS C:\Users\<Anmol> > javac exe5_6.java
PS C:\Users\<Anmol> > java exe5_6
Method with float and int parameters called: 10.6, 10
Method with int and float parameters called: 10, 10.8
The call obj.display(10, 10) is ambiguous because both methods can be called with the given arguments. The compiler cannot determine which
method to call since both methods can accept the arguments (10 can be treated as an int or a float). To resolve this ambiguity, you can eit
her cast the arguments to the desired types or provide different argument values that clearly match one of the method signatures.
PS C:\Users\<Anmol> >
```

Observation: The call obj.display(10, 10) is ambiguous because both methods can be called with the given arguments. The compiler cannot determine which method to call since both methods can accept the arguments (10 can be treated as an int or a float). To resolve this ambiguity, you can either cast the arguments to the desired types or provide different argument values that clearly match one of the method signatures.

Exe5_7

```
// Create a class with two instance variables and one static variable.
// Modify all variables from main() using object reference.
// Create two objects and observe the changes.
// Explain why static variable changes affect all objects
class ex
{
    int v1;
    int v2;
```

```java
    static int sv;
    void modify(int var1,int var2,int stvar)
    {
        v1=var1;
        v2=var2;
        sv=stvar;
    }
    void display()
    {
        System.out.println("Instance Variable 1: "+v1);
        System.out.println("Instance Variable 2: "+v2);
        System.out.println("Static Variable: "+sv);
    }
}
public class exe5_7
{
    public static void main(String args[])
    {
        ex obj1=new ex();
        ex obj2=new ex();
        obj1.modify(3,7,17);
        obj2.modify(96,25,19);
        System.out.println("Object 1:");
        obj1.display();
        System.out.println("Object 2:");
        obj2.display();
    }
}
```

```
OUTPUT 67    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe5_7.java
PS C:\Users\<Anmol> > java exe5_7
Object 1:
Instance Variable 1: 3
Instance Variable 2: 7
Static Variable: 19
Object 2:
Instance Variable 1: 96
Instance Variable 2: 25
Static Variable: 19
PS C:\Users\<Anmol> >
```

Observation: The static variable 'sv' is shared among all instances of the class 'ex'. When we modify 'sv' using one object (obj1), it changes for all objects of the class, including obj2. This is because static variables belong to the class

rather than any specific instance, so they are shared across all instances. Therefore, when obj2 modifies 'sv', it overwrites the value set by obj1, and both obj1 and obj2 will reflect the latest value of 'sv'.

Exe5_8

```
// Create a class where constructor calls an instance method.
// Instance method reads input using BufferedReader and adds value to instance
// variable.
// Create object from main() and observe execution
import java.io.*;
class Constructor
{
    int a;
    Constructor()
    {
        readValue();
    }
    void readValue()
    {
        {
            try
            {
                BufferedReader br=new BufferedReader(new
InputStreamReader(System.in));
                System.out.print("Enter value: ");
                int x=Integer.parseInt(br.readLine());
                a=a+x;
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
}
public class exe5_8
{
    public static void main(String args[])
    {
        Constructor obj=new Constructor();
        System.out.println("Value of a: "+obj.a);
    }
}
```

```
PS C:\Users\<Anmol> > javac exe5_8.java
PS C:\Users\<Anmol> > java exe5_8
Enter value: 7
Value of a: 7
PS C:\Users\<Anmol> >
```

Observation: When the object of the Constructor class is created, the constructor is called, which in turn calls the instance method readValue(). The readValue() method prompts the user to enter a value, reads it from the console, and adds it to the instance variable 'a'. Finally, the value of 'a' is displayed in the main method.

Exe5_9

```java
// Create a class with no constructor and one instance variable.
// Create object and print variable value.
// Then define your own constructor.
// Observe difference and explain default constructor role.
class nocon
{
    int a;
    // nocon(int x)
    // {
    //     a=x;
    // }
}
class exe5_9
{
    public static void main(String args[])
    {
        nocon obj=new nocon();
        System.out.println("Value of a: "+obj.a);
        // nocon obj1=new nocon(17);
        // System.out.println("Value of a: "+obj1.a);
        System.out.println("In this program if we comment in the comment out
part of both the class and main method then the default constructor will be
called and the value of a will be 0 because it is an instance variable and it
is initialized to 0 by default. But if we uncomment the comment out part then
the default constructor will not be called and the value of a will be 17
because we have defined our own constructor which takes an integer argument
and assigns it to a.");
    }
}
```

```
PS C:\Users\<Anmol> > java exe5_9
Value of a: 0
In this program if we comment in the comment out part of both the class and main method then the default constructor will be called and the
 value of a will be 0 because it is an instance variable and it is initialized to 0 by default. But if we uncomment the comment out part th
en the default constructor will not be called and the value of a will be 17 because we have defined our own constructor which takes an inte
ger argument and assigns it to a.
PS C:\Users\<Anmol> >
```

Observation: In this program, if we comment out the constructor in the nocon class and the corresponding object creation in the main method, the default constructor will be called when we create an object of the nocon class. The instance variable 'a' will be initialized to its default value, which is 0 for integers. However, if we uncomment the constructor and create an object using it, the default constructor will not be called, and instead, our defined constructor will be executed, assigning the value 17 to 'a'. This demonstrates that when a user-defined constructor is present, the default constructor is not automatically provided by the compiler.

Exe5_10

```java
// Create a class Student with name, roll number and static variable
totalStudents.
// Overload constructors (default and parameterized).
// Each time object is created, increment totalStudents.
// Display total number of students created.
class Student
{
    String name;
    int rollno;
    static int totalStudents;
    Student()
    {
        totalStudents++;
    }
    Student(String name, int rollno)
    {
        this.name=name;
        this.rollno=rollno;
        totalStudents++;
    }
}
public class exe5_10
{
    public static void main(String args[])
    {
        Student s1=new Student();
        Student s2=new Student("Alice", 1);
        Student s3=new Student("Bob", 2);
```

```
        System.out.println("Total students created: "+Student.totalStudents);
    }
}
```

```
PS C:\Users\<Anmol> > javac exe5_10.java
PS C:\Users\<Anmol> > java exe5_10
Total students created: 3
PS C:\Users\<Anmol> >
```

Observation: In this program, we have a Student class with two constructors: a default constructor and a parameterized constructor. Each time an object of the Student class is created, the static variable totalStudents is incremented. In the main method, we create three Student objects, which results in totalStudents being incremented three times. Finally, we display the total number of students created, which will output "Total students created: 3".

Exe5_11

```
// Create a class to overload method area() for circle, rectangle, and
triangle.
import java.util.*;
class Area
{
    double area(double r)
    {
        return 3.14*r*r;
    }
    double area(double l,double b)
    {
        return l*b;
    }
    double area(double a,double b,double c)
    {
        double s=(a+b+c)/2;
        return Math.sqrt(s*(s-a)*(s-b)*(s-c));
    }
}
public class exe5_11
{
    public static void main(String args[])
    {
        Scanner sc=new Scanner(System.in);
        Area obj=new Area();
        System.out.println("Enter radius of circle: ");
```

```java
        double r=sc.nextDouble();
        System.out.println("Area of circle: "+obj.area(r));
        System.out.println("Enter length and breadth of rectangle: ");
        double l=sc.nextDouble();
        double b=sc.nextDouble();
        System.out.println("Area of rectangle: "+obj.area(l,b));
        System.out.println("Enter sides of triangle: ");
        double x=sc.nextDouble();
        double y=sc.nextDouble();
        double z=sc.nextDouble();
        System.out.println("Area of triangle: "+obj.area(x,y,z));
    }
}
```

```
OUTPUT 73    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe5_11.java
PS C:\Users\<Anmol> > java exe5_11
Enter radius of circle:
5.9
Area of circle: 109.30340000000002
Enter length and breadth of rectangle:
7.8
6.9
Area of rectangle: 53.82
Enter sides of triangle:
5
6
7
Area of triangle: 14.696938456699069
PS C:\Users\<Anmol> >
```

Observation: The program defines a class Area with three overloaded methods named area() to calculate the area of a circle, rectangle, and triangle. The main method prompts the user to input the necessary dimensions for each shape, calls the appropriate area() method based on the input, and displays the calculated area for each shape.

Exe5_12

```java
// Implement constructor overloading in a Student class.

class student
{
    String name;
```

```
    int roll;
    student()
    {
        name="Rahul";
        roll=35;
    }
    student(String name,int roll)
    {
        this.name=name;
        this.roll=roll;
    }
}
public class exe5_12
{
    public static void main(String args[])
    {
        student s1=new student();
        student s2=new student("Anmol", 17);
        System.out.println("Student 1: Name="+s1.name+",Roll="+s1.roll);
        System.out.println("Student 2: Name="+s2.name+",Roll="+s2.roll);
    }
}
```

```
OUTPUT 73    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe5_12.java
PS C:\Users\<Anmol> > java exe5_12
Student 1: Name=Rahul,Roll=35
Student 2: Name=Anmol,Roll=17
PS C:\Users\<Anmol> >
```

Observation: The program demonstrates constructor overloading in the Student class. The default constructor initializes the name and roll number with fixed values, while the parameterized constructor allows for user-provided values. In the main method, we create two Student objects using both constructors and display their details.

Exe5_13

```
// Overload a method to calculate volume of cube and cylinder.
import java.util.*;
class Volume
{
    double volume(double a)
```

```java
    {
        return a*a*a;
    }
    double volume(double r,double h)
    {
        return 3.14*r*r*h;
    }
}
public class exe5_13
{
    public static void main(String args[])
    {
        Scanner sc=new Scanner(System.in);
        Volume obj=new Volume();
        System.out.println("Enter side of cube: ");
        double a=sc.nextDouble();
        System.out.println("Volume of cube: "+obj.volume(a));
        System.out.println("Enter radius and height of cylinder: ");
        double r=sc.nextDouble();
        double h=sc.nextDouble();
        System.out.println("Volume of cylinder: "+obj.volume(r,h));
    }
}
```

```
PS C:\Users\<Anmol> > javac exe5_13.java
PS C:\Users\<Anmol> > java exe5_13
Enter side of cube:
6.9
Volume of cube: 328.50900000000007
Enter radius and height of cylinder:
9.6
5.3
Volume of cylinder: 1533.7267199999997
PS C:\Users\<Anmol> >
```

Observation: The program defines a class Volume with two overloaded methods named volume() to calculate the volume of a cube and a cylinder. The main method prompts the user to input the necessary dimensions for each shape, calls the appropriate volume() method based on the input, and displays the calculated volume for each shape.

Exe5_14

```java
// Create a class demonstrating method overloading using different data types.
```

```java
class Overloading
{
    void display(int a)
    {
        System.out.println("Integer: "+a);
    }
    void display(double b)
    {
        System.out.println("Double: "+b);
    }
    void display(String c)
    {
        System.out.println("String: "+c);
    }
}
public class exe5_14
{
    public static void main(String args[])
    {
        Overloading obj=new Overloading();
        obj.display(17);
        obj.display(3.14);
        obj.display("Hello");
    }
}
```

```
PS C:\Users\<Anmol> > javac exe5_14.java
PS C:\Users\<Anmol> > java exe5_14
Integer: 17
Double: 3.14
String: Hello
PS C:\Users\<Anmol> >
```

Observation: The program defines a class Overloading with three overloaded methods named display() that accept different data types: int, double, and String. In the main method, we create an instance of the Overloading class and call the display() method with different types of arguments (an integer, a double, and a string). The appropriate overloaded method is called based on the type of argument passed, demonstrating method overloading in Java.

Exe5_15

```java
// Write a program showing constructor chaining.
class Student
{
    String name;
    int rollno;
    Student()
    {
        this("Rahul",35);
    }
    Student(String name,int rollno)
    {
        this.name=name;
        this.rollno=rollno;
    }
}
public class exe5_15
{
    public static void main(String args[])
    {
        Student s1=new Student();
        Student s2=new Student("Anmol",37);
        System.out.println("Student 1: Name="+s1.name+",Roll="+s1.rollno);
        System.out.println("Student 2: Name="+s2.name+",Roll="+s2.rollno);
    }
}
```

```
PS C:\Users\<Anmol> > javac exe5_15.java
PS C:\Users\<Anmol> > java exe5_15
Student 1: Name=Rahul,Roll=35
Student 2: Name=Anmol,Roll=37
PS C:\Users\<Anmol> >
```

Observation: The program demonstrates constructor chaining in the Student class. The default constructor calls the parameterized constructor using the 'this' keyword, passing fixed values for name and roll number. When we create an object using the default constructor, it initializes the name and roll number with the values specified in the parameterized constructor. When we create an object using the parameterized constructor, it initializes the name and roll number with the user-provided values. Finally, we display the details of both students to verify that the constructor chaining works as expected.